PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

Rule-Based Automated Branch Synchronization and Merge Governance System with Compliance Audit Engine

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Simran

2210992382

Supervised By:

Prof. Dr. S. B. Goyal

Dean,

Department of Computer Science & Engineering

Chitkara University,

Punjab, India

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
1.	Declaration	3
2.	Acknowledgement	4
3.	List of Figures and Tables	5
4.	Abstract	6
5.	Introduction	7
6.	Methodology	8
7.	Tools and Technologies	12
8.	Implementation	13
9.	Major Findings/Outcomes/Output/Results	15
10.	Conclusion and Future Scope	25
11.	References	26
12.	Appendices	27

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **Rule-Based Automated Branch Synchronization and Merge Governance System with Compliance Audit Engine** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Prof. Dr. S. B. Goyal**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **Simran**

Date: 01-05-2026 University Roll No. 2210992382

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Prof. Dr. S. B. Goyal

Dean

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Prof. Dr. S. B. Goyal**, for his invaluable guidance, continuous support and insightful suggestions throughout the development of this project. His expertise, encouragement and timely feedback have been instrumental in shaping this work and in successfully completing the project.

I also extend my heartfelt thanks to **Chitkara University, Rajpura**, for providing the necessary infrastructure, resources and a conducive academic environment that enabled me to carry out this project efficiently.

I would like to acknowledge the support and guidance provided by the faculty members of the Department of Computer Science and Engineering, whose knowledge and assistance contributed significantly during the course of this work.

Furthermore, I am grateful to my peers and colleagues for their cooperation, constructive discussions and support, which helped me in overcoming various challenges during the project.

Finally, I express my deep gratitude to my family for their constant encouragement, motivation and moral support, which played an important role in the successful completion of this project.

List of Figures and Tables

List of Figures

Figure No.	Title	Page No.
Figure 1	System Architecture Diagram	16
Figure 2	Workflow Diagram	17
Figure 3	Branch Hierarchy (Allowed Operations)	18
Figure 4	Branch Hierarchy (Not Allowed Operations)	18
Figure 5	Compliance Monitoring Flow	19
Figure 6	Merge Conflict Detection and Resolution Flow	20
Figure 7	Sequence Diagram	22
Figure 8	Prototype Flow Diagram	23

Abstract

In modern software development, distributed version control systems play a critical role in enabling collaboration across multiple teams working on shared codebases. Most development workflows adopt a multi-branch strategy consisting of development, sprint and release branches to manage code stability and deployment cycles. However, in real-world scenarios, improper adherence to branching strategies, unauthorised commits and lack of synchronization between branches often lead to branch divergence, increased merge conflicts and inconsistent code states. These issues significantly impact development efficiency and require considerable manual effort for resolution.

This project presents a **Rule-Based Automated Branch Synchronization and Merge Governance System with a Compliance Audit Engine**, designed to address the limitations of existing version control practices. The proposed system introduces a structured and automated mechanism that enforces downward branch synchronization, ensuring that validated code changes from the release branch are consistently propagated to downstream branches such as sprint and development. The system operates based on a configurable rule engine that defines merge direction, synchronization intervals, conflict resolution strategies and backup preferences.

A key contribution of this work is the integration of a compliance audit engine that continuously monitors commit activities across branches and identifies violations such as out-of-order commits and unauthorised modifications. These violations are recorded along with detailed metadata, including commit identifiers, developer information and timestamps, thereby enabling transparency, accountability and traceability within the development process. Additionally, the system incorporates conflict prediction and detection mechanisms to identify potential merge conflicts in advance and apply predefined resolution strategies, reducing manual intervention.

The proposed solution has been implemented and validated using Git-based repositories structured with multiple branches in a controlled development environment. Experimental evaluation demonstrates that the system effectively automates branch synchronization, detects inconsistencies and significantly reduces merge conflict occurrences. Furthermore, it minimises manual synchronization efforts and enhances overall code reliability.

By combining rule-based automation, branch governance and compliance monitoring into a unified platform, this system provides a scalable and efficient solution for modern software development environments. It is particularly suitable for Agile methodologies and enterprise-level projects, where maintaining code consistency and reducing development overhead are critical for successful software delivery.

Introduction

In the era of modern software engineering, distributed version control systems such as Git have become an integral part of collaborative development. These systems enable multiple developers and teams to work simultaneously on a shared codebase by maintaining different branches for development, testing and release purposes. A typical branching strategy includes development, sprint and release branches, where code flows progressively from development to release after validation and testing.

Despite the structured nature of these workflows, real-world development environments often face several challenges due to improper adherence to branching strategies. Developers may introduce changes directly into higher-level branches such as sprint or release, bypassing the intended development flow. Additionally, code changes made in the release branch are not always propagated back to the lower branches, leading to inconsistencies across the repository. These practices result in branch divergence, frequent merge conflicts, redundant efforts and increased risk of errors in future releases.

Merge conflicts, in particular, pose a significant challenge in collaborative environments. Resolving these conflicts requires manual intervention, consumes valuable development time and may lead to incorrect conflict resolution if not handled carefully. Studies indicate that the time required to resolve merge conflicts can vary significantly depending on the complexity of the codebase, number of developers involved and the extent of conflicting changes. This highlights the need for a more systematic and automated approach to manage branch synchronization and conflict resolution.

Existing solutions, such as manual synchronization, scheduled merge scripts, access control policies and continuous integration pipelines, address specific aspects of the problem but fail to provide a comprehensive solution. They lack mechanisms for enforcing branch hierarchy, detecting unauthorized commits and maintaining consistency across all branches in an automated manner.

To overcome these limitations, this project proposes a **Rule-Based Automated Branch Synchronization and Merge Governance System with a Compliance Audit Engine**. The system is designed to enforce a structured branching strategy by automating downward synchronization of code changes from higher-level branches to lower-level branches. It also introduces a rule-based governance mechanism to control merge directions and a compliance audit engine to monitor and record violations in commit activities.

The proposed system not only reduces manual effort but also improves code consistency, enhances developer accountability and minimizes merge conflicts. By integrating synchronization, governance and compliance monitoring into a single platform, the solution aims to streamline version control workflows and improve overall development efficiency in both Agile and enterprise environments.

Methodology

The proposed system follows a structured and rule-based methodology to automate branch synchronization and enforce governance in version-controlled repositories. The methodology is designed to ensure consistency across multiple branches, reduce manual intervention and provide transparency in commit activities through a compliance audit mechanism.

The overall approach involves configuration-driven execution, periodic synchronization cycles, commit analysis, conflict detection and audit logging. The system operates as a combination of multiple logical engines, each responsible for a specific function within the synchronization and governance process.

---

1. System Configuration

The methodology begins with the initialization of the system through a user-defined configuration module. This module allows users to specify key parameters required for automated synchronization, including:

- Source branch (e.g., release or sprint)
- Target branch (e.g., sprint or development)
- Synchronization interval and number of cycles
- Conflict resolution preferences
- Backup branch creation option

This configuration ensures flexibility and adaptability of the system to different project requirements and workflows.

---

2. Scheduling and Execution of Synchronization Cycles

Once the configuration is set, the scheduler engine activates the synchronization process at predefined intervals. The system performs synchronization in cycles, ensuring continuous monitoring and updating of branch states.

During each cycle, the system verifies whether the merge operation follows the allowed branch hierarchy. Only downward merges (e.g., release to sprint, sprint to development) are permitted, while unauthorized merge directions are blocked.

---

3. Commit Analysis and Validation

The commit analysis engine plays a crucial role in examining the differences between source and target branches. It compares commit histories using version control commands to identify:

- Missing commits in target branches
- Extra or unauthorized commits in higher-level branches

This step ensures that the system detects inconsistencies and maintains proper commit flow across branches.

---

4. Compliance Monitoring and Violation Detection

The compliance audit engine monitors commit activities to detect violations of branching rules. When unauthorized or out-of-order commits are identified, the system records detailed metadata, including:

- Commit ID
- Developer identity
- Commit message
- Timestamp
- Branch where violation occurred

These records are maintained for audit purposes and are later displayed through a compliance dashboard, ensuring accountability and traceability.

---

5. Backup Creation

Before initiating the merge process, the system optionally creates a backup of the target branch. This backup mechanism ensures that the original state of the branch can be restored in case of unexpected errors or conflicts during synchronization.

---

6. Conflict Prediction and Detection

Prior to merging, the system analyzes file-level differences between branches to identify potential merge conflicts. This proactive detection helps in:

- Reducing unexpected failures during merging
- Preparing the system for conflict resolution

The system flags risky scenarios where multiple developers have modified the same lines of code.

---

7. Rule-Based Merge Governance

The merge governance engine enforces predefined rules to control branch synchronization. It ensures that only valid merge directions are allowed and prevents unauthorized operations.

Allowed operations:

- Release → Sprint
- Release → Development

- Sprint → Development

Restricted operations:

- Development → Sprint
- Development → Release
- Sprint → Release

This rule-based approach ensures adherence to proper branching strategies.

---

8. Conflict Resolution Mechanism

In cases where conflicts are detected, the system applies a predefined conflict resolution strategy based on user configuration. The system can:

- Prefer source branch changes
- Prefer target branch changes

If conflicts cannot be resolved automatically, the system aborts the merge process and logs the issue for manual intervention.

---

9. Automated Branch Synchronization

After validation and conflict handling, the system performs the merge operation to synchronize branches. This ensures that validated code changes are propagated consistently across downstream branches.

---

10. Logging and Reporting

All activities performed during synchronization cycles are recorded by the logging engine. The system maintains detailed logs of:

- Synchronization events
- Detected violations
- Merge outcomes
- Conflict occurrences
- Backup operations

These logs are used to generate reports and provide insights into system performance and developer activity.

---

11. Output and Visualization

The final step involves presenting the results through a compliance audit dashboard. The dashboard provides:

- Violation reports
- Synchronization status
- Conflict insights
- Developer activity tracking

This enhances transparency and helps teams improve their development practices.

Summary of Methodology

The methodology integrates configuration, automation, validation and monitoring into a unified workflow. By combining rule-based synchronization with compliance auditing, the system ensures consistent code propagation, reduces merge conflicts and enhances overall development efficiency.

Tools and Technologies

The proposed system is developed using a combination of version control tools, programming technologies and automation utilities to ensure efficient branch synchronization, governance and compliance monitoring.

Git (Version Control System):
Git is the core technology used for managing repositories and handling multiple branches such as release, sprint and development. It is used for tracking code changes, performing merges and analysing commit histories. Commands like git log, git diff and git merge are used for synchronization and conflict detection.

GitHub (Repository Hosting Platform):
GitHub is used to host and manage the project repository. It enables collaboration, maintains version history and supports integration with automation workflows.

Programming Language (Python / Shell Scripting):
The rule-based automation and synchronization logic are implemented using Python or Shell scripting. These are used to automate merge operations, process commit data and implement governance rules.

Visual Studio Code (IDE):
Visual Studio Code is used as the development environment for writing and executing scripts. It provides useful extensions and tools that improve productivity and simplify debugging.

Scheduler (Cron Jobs / Task Scheduler):
A scheduling mechanism is used to automate synchronization cycles at predefined intervals, ensuring continuous execution without manual intervention.

Logging and Audit Mechanism:
A logging system is implemented to record synchronization activities, detect violations and maintain audit trails for accountability and transparency.

Diagramming Tools:
Tools such as Draw.io or Lucidchart are used to create system architecture, workflow and sequence diagrams for better visualisation of the proposed system.

Implementation

The proposed system is implemented using a Git-based repository structured into three primary branches, namely release, sprint and development, to simulate a real-world software development workflow. The implementation focuses on automating branch synchronization, enforcing branch hierarchy rules and maintaining compliance through continuous monitoring.

The system begins with a configuration setup where the user defines parameters such as source branch, target branch, synchronization interval, number of cycles, conflict resolution preference and backup option. These parameters control the behaviour of the system and allow flexibility based on different project requirements.

A scheduling mechanism is used to automate the synchronization process. Based on the defined interval, the system triggers synchronization cycles periodically without requiring manual intervention. During each cycle, the system first validates whether the merge direction follows the predefined branch hierarchy rules. Only downward merge operations, such as release to sprint and sprint to development, are allowed, while unauthorised merge attempts are blocked and recorded.

The implementation uses git commands such as git log and git diff to perform commit analysis between branches. This helps in identifying differences in commit histories and detecting out-of-order or unauthorised commits. When such violations are found, the system logs relevant details, including commit ID, developer name, commit message and timestamp for audit purposes.

Before performing the merge operation, the system optionally creates a backup branch from the target branch. This ensures that the original state of the branch can be restored in case of unexpected errors during synchronization.

The system also includes a conflict detection mechanism that analyses file-level changes between branches to identify potential merge conflicts. In case conflicts are detected, the system applies a predefined conflict resolution strategy based on user input. If conflicts cannot be resolved automatically, the merge process is aborted, and the issue is logged for manual resolution.

Once all validations are completed, the system performs the merge operation to synchronize the branches. This ensures that validated code from higher-level branches is consistently propagated to lower-level branches, maintaining code consistency and reducing branch divergence.

All activities performed during the synchronization process are recorded using a logging mechanism. The system maintains logs for synchronization events, detected violations, conflict details and backup operations. These logs are further used for audit and monitoring purposes.

The implementation is tested using multiple scenarios, including normal synchronization, detection of unauthorised commits and handling of merge conflicts. The results demonstrate that the system successfully automates branch synchronization, reduces manual effort and improves overall development efficiency.

Pseudocode/Algorithm

Algorithm (Pseudo-code)

```
INPUT:
  source_branch = master     //Either master or sprint as downward merge is allowed.
  target_branch = sprint     //Either sprint or dev
  no_of_cycle = 1            //How many cycles for sync
  sync_interval = 2  hrs     //After how much time each cycle should start sync again
  back_up = yes              //Either yes/no to create a backup branch or not
  conflict_resolution = master     //Which branch changes to keep in case of merge conflict if any
FOR each no_of_cycle in sync_interval:
        FOR each branch pair (source_branch -> target_branch):
                IF (source -> target) is allowed:
                        commits_diff = git_log(source_branch NOT target_branch)
                        IF commits_diff is not empty:
                                violations = git_log(target_branch NOT source_branch)
                                IF violations exist:
                                        Log violation with metadata
                                conflicts = detect_conflicts(source_branch, target_branch)
                                IF conflicts == TRUE:
                                        APPLY conflict_resolution strategy
                                                IF unresolved:
                                                        ABORT merge
                                                        LOG issue
                                                ELSE:
                                                        MERGE
                                ELSE:
                                        MERGE automatically
                                LOG Success
                ELSE:
                        BLOCK operation
                        LOG unauthorized attempt
EXIT()
```

Major Findings/Outcomes/Output/Results

• Automated Branch Synchronization:
The system successfully performed automatic downward synchronization from release → sprint → development branches at scheduled intervals without manual intervention.

• Detection of Unauthorized Commits:
Out-of-order commits (direct commits to sprint/release) were correctly identified and flagged during execution.

• Compliance Audit Logging:
All violations were logged with details such as commit ID, developer name, message and timestamp, ensuring proper tracking and accountability.

• Merge Conflict Detection:
The system detected potential merge conflicts before synchronization using file-level comparison.

• Conflict Handling:
Predefined conflict resolution strategies were applied. In unresolved cases, the merge was aborted and logged for manual resolution.

• Backup Mechanism:
Backup branches were successfully created before synchronization, enabling safe rollback in case of errors.

• Enforcement of Branch Rules:
Only allowed merge directions (release → sprint → development) were executed. Invalid merge attempts were blocked.

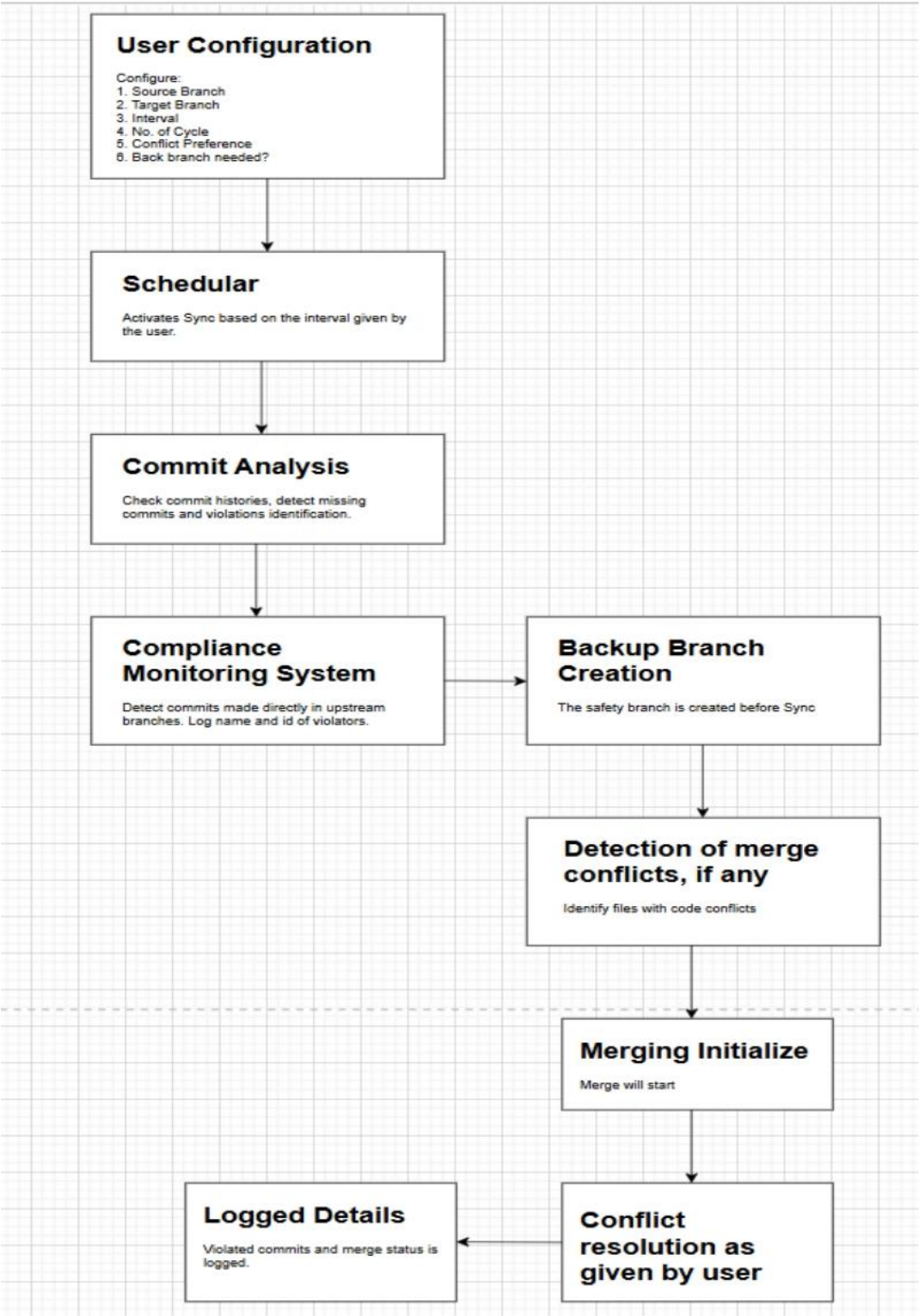
• Reduction in Manual Effort:
Manual synchronization effort was reduced significantly as the process was automated.

• Improvement in Code Consistency:
Continuous synchronization ensured consistent code across all branches and reduced branch divergence.

• Overall System Performance:
The system worked efficiently across multiple test scenarios and demonstrated reliable automation of branch governance and synchronization.

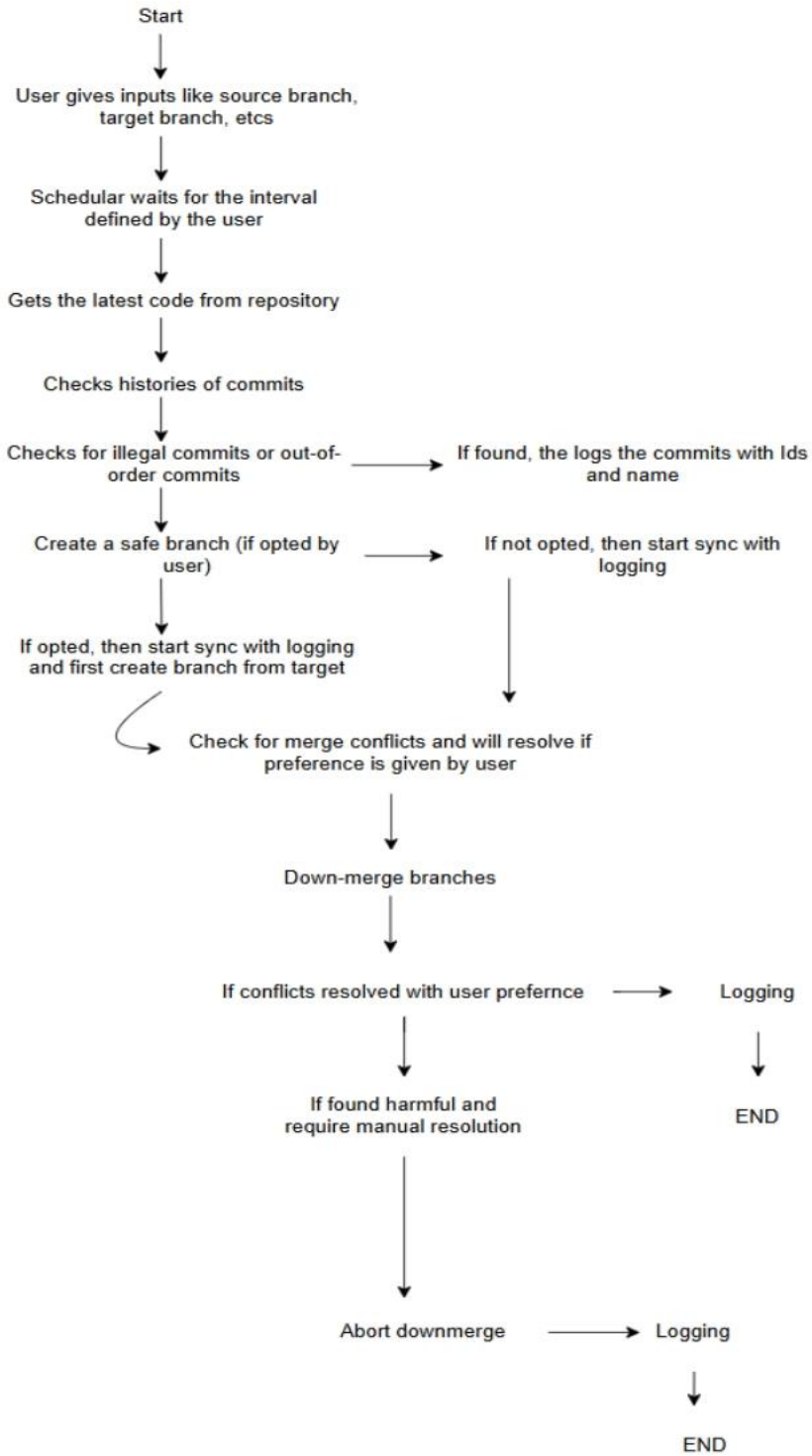
1. System Architecture Diagram:

This diagram illustrates the complete system flow from user input to audit logging and successful sync engines between branches.



2. Workflow Diagram:

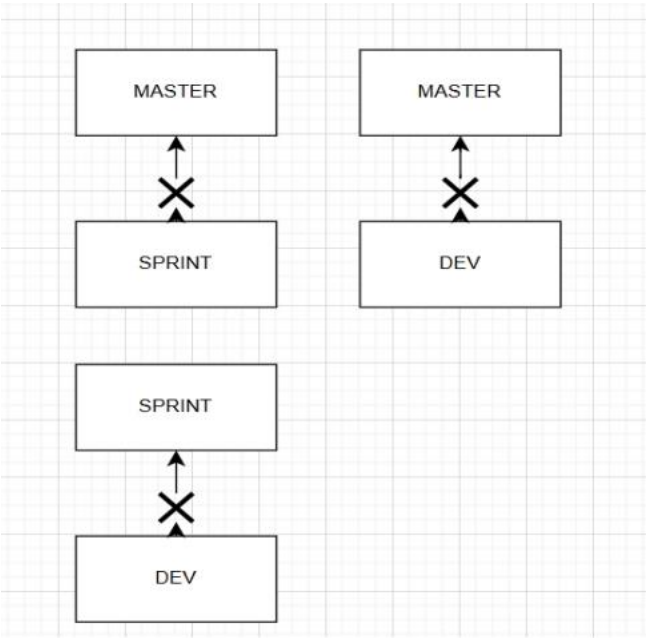
This diagram illustrates the step-by-step operational workflow through each engine, with a negate statement flow and true flow between engines.



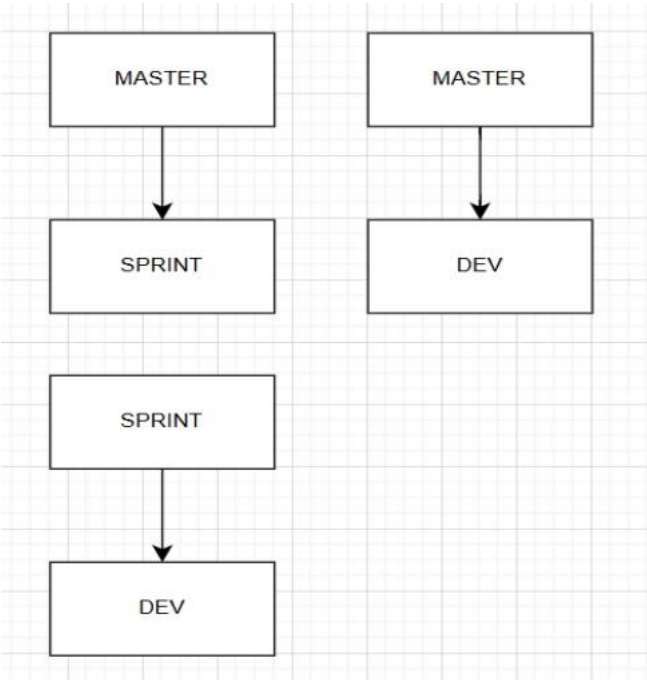
3. Branch hierarchy:

These two diagrams show the branch hierarchy supported for Rule-Based Automated Synchronization to happen between branches.

NOT ALLOWED

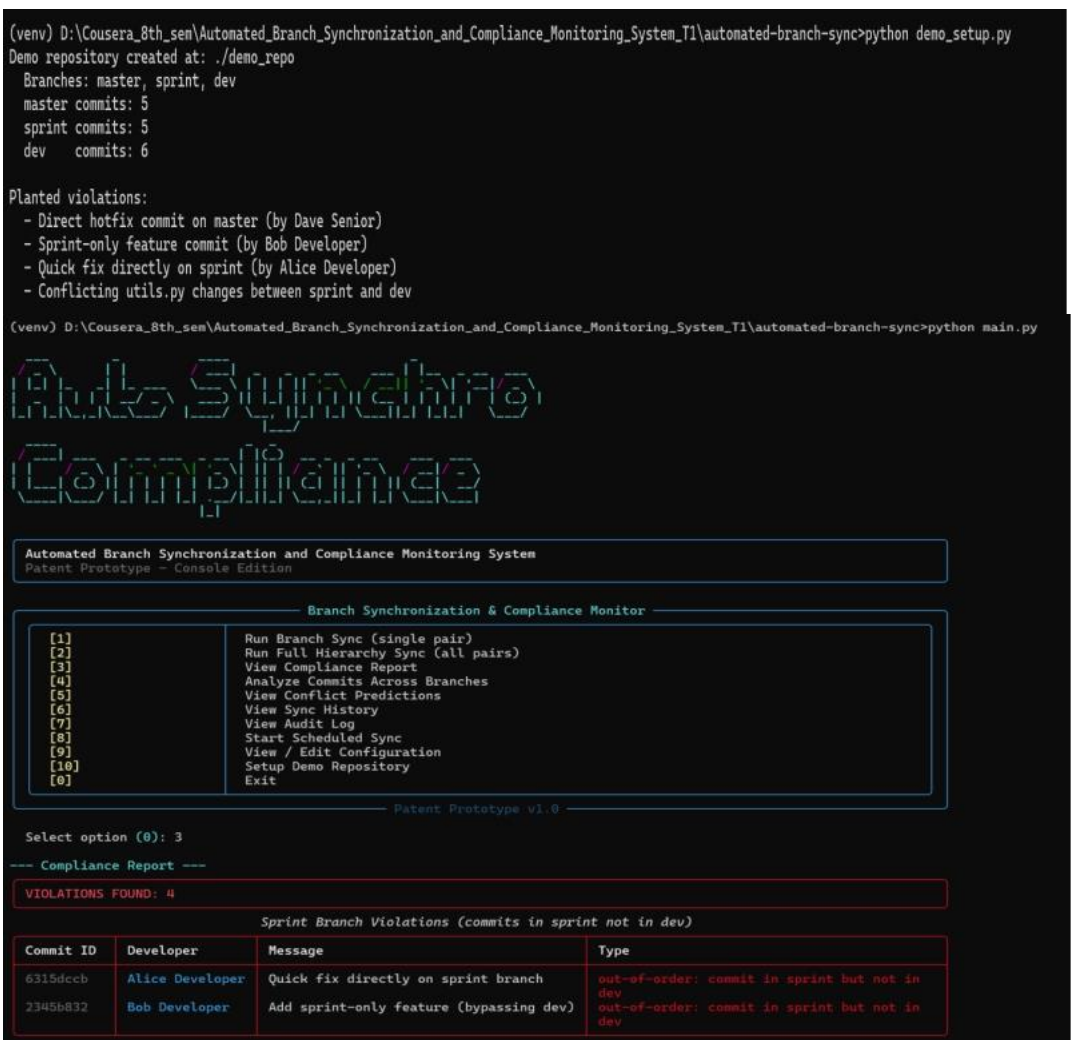


ALLOWED



4. Compliance Monitoring Flow:

This diagram and screenshot will check the developer out of order commits.



5. Merge Conflict Detection and Resolution Flow:

This diagram checks the conflicts and asks the user which strategy to follow for merge conflicts resolution, if any. The screenshots are attached for this from the mini prototype.

```
Start Merge Process
      |
      v
Compare Files Between Branches
      |
      v
Same File Modified?
      |
 +----+----+
 |         |
Yes        No
 |         |
Detect Conflict
 |         |
Ask Merge Strategy
 |         |
Prefer Source Branch?
 |         |
 +----+----+
 |         |
Yes        No
 |         |
Keep Source   Keep Target
Changes       Changes
 |
 v
Resolve Conflict
 |
 v
Complete Merge
 |
 v
Log Merge Result
```

Screenshots merge conflict detection and resolution engine

```
Branch Synchronization & Compliance Monitor

[1]   Run Branch Sync (single pair)
[2]   Run Full Hierarchy Sync (all pairs)
[3]   View Compliance Report
[4]   Analyze Commits Across Branches
[5]   View Conflict Predictions
[6]   View Sync History
[7]   View Audit Log
[8]   Start Scheduled Sync
[9]   View / Edit Configuration
[10]  Setup Demo Repository
[0]   Exit

Patent Prototype v1.0

Select option (0): 2

--- Full Hierarchy Sync ---
    Sync Pairs

 #   Source   Target
 1   master   sprint
 2   master   dev
 3   sprint   dev

Proceed with hierarchy sync? [y/n] (y): y

Sync Result
Source: master
Target: sprint
Status: SYNCED

Commit Analysis: 0 source-only, 3 target-only
Compliance: 5 violation(s)
Backup: backup/sprint_20260318_000446
Conflict Prediction: 0 file(s)
Merge: success - Successfully merged master into sprint

Sync Result
Source: master
Target: dev
Status: SYNCED

Commit Analysis: 2 source-only, 3 target-only
Compliance: 5 violation(s)
Backup: backup/dev_20260318_000447
Conflict Prediction: 0 file(s)
```

```
Sync Result
Source: master
Target: dev
Status: SYNCED

Commit Analysis: 2 source-only, 3 target-only
Compliance: 5 violation(s)
Backup: backup/dev_20260318_000447
Conflict Prediction: 0 file(s)
Merge: success - Successfully merged master into dev

Sync Result
Source: sprint
Target: dev
Status: SYNCED_WITH_RESOLUTION

Commit Analysis: 3 source-only, 4 target-only
Compliance: 3 violation(s)
Backup: backup/dev_20260318_000448
Conflict Prediction: 1 file(s)
Merge: conflict - Merge conflicts detected in 1 file(s)
Resolution: resolved - All 1 conflict(s) resolved using source strategy
```

6. Sequence Diagram

This diagram shows the interaction between different system components during sync cycle.

```
User
 |
 | Configure Settings
 v
Scheduler
 |
 | Trigger Sync
 v
Commit Analyzer
 |
 | Compare Branches
 v
Compliance Monitor
 |
 | Detect Violations
 v
Backup Module
 |
 | Create Backup Branch
 v
Conflict Detector
 |
 | Check File Changes
 v
Merge Engine
 |
 | Perform Downmerge
 v
Conflict Resolver
 |
 | Apply Merge Strategy
 v
Logging System
 |
 | Store Logs
 v
End
```

7. Prototype Flow

This diagram represents the implementation level flow of the prototype and proposed solution.

```
Run Python Script
      |
      v
Enter Source Branch
      |
      v
Enter Target Branches
      |
      v
Enter Sync Interval
      |
      v
Enter Number of Cycles
      |
      v
Choose Conflict Strategy
      |
      v
Start Automation
      |
      v
Check Branch Differences
      |
      v
Log Compliance Violations
      |
      v
Create Backup Branch
      |
      v
Perform Downmerge
      |
      v
Log Results
      |
      v
Repeat Until Cycles Complete
```

8. Test Cases Present in the Mini Prototype (52 test cases)

```
(venv) D:\Cousera_8th_sem\Automated_Branch_Synchronization_and_Compliance_Monitoring_System_T1\automated-branch-sync>python -m pytest tests/ -v --tb=short
============================================= test session starts =============================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0 -- D:\Cousera_8th_sem\Automated_Branch_Synchronization_and_Compliance_Monitoring_System_T1\automated-branch-sync\venv\S
cripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Cousera_8th_sem\Automated_Branch_Synchronization_and_Compliance_Monitoring_System_T1\automated-branch-sync
collected 52 items

tests/test_backup.py::TestBackup::test_create_backup_branch PASSED                                  [  1%]
tests/test_backup.py::TestBackup::test_backup_preserves_state PASSED                                 [  3%]
tests/test_backup.py::TestBackup::test_list_backups PASSED                                           [  5%]
tests/test_backup.py::TestBackup::test_rollback PASSED                                               [  7%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_get_branch_commits PASSED                   [  9%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_identical_branches_no_diff PASSED           [ 11%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_diverged_branches PASSED                    [ 13%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_compare_branches PASSED                     [ 15%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_commit_metadata PASSED                      [ 17%]
tests/test_compliance.py::TestCompliance::test_compliant_branches PASSED                            [ 19%]
tests/test_compliance.py::TestCompliance::test_sprint_violation_detected PASSED                     [ 21%]
tests/test_compliance.py::TestCompliance::test_master_violation_detected PASSED                     [ 23%]
tests/test_compliance.py::TestCompliance::test_full_compliance_report PASSED                        [ 25%]
tests/test_compliance.py::TestCompliance::test_violations_logged_to_db PASSED                       [ 26%]
tests/test_conflict_detector.py::TestConflictDetector::test_no_conflicts_identical_branches PASSED  [ 28%]
tests/test_conflict_detector.py::TestConflictDetector::test_no_conflicts_clean_divergence PASSED    [ 30%]
tests/test_conflict_detector.py::TestConflictDetector::test_conflict_detected PASSED                 [ 32%]
tests/test_conflict_detector.py::TestConflictDetector::test_conflict_risk_level PASSED               [ 34%]
tests/test_conflict_resolver.py::TestConflictResolver::test_resolve_with_source_priority PASSED      [ 36%]
tests/test_conflict_resolver.py::TestConflictResolver::test_resolve_with_target_priority PASSED      [ 38%]
tests/test_conflict_resolver.py::TestConflictResolver::test_no_conflicts_returns_clean PASSED        [ 40%]
tests/test_merge_engine.py::TestMergeEngine::test_master_to_sprint_allowed PASSED                    [ 42%]
tests/test_merge_engine.py::TestMergeEngine::test_master_to_dev_allowed PASSED                       [ 44%]
tests/test_merge_engine.py::TestMergeEngine::test_sprint_to_dev_allowed PASSED                       [ 46%]
tests/test_merge_engine.py::TestMergeEngine::test_dev_to_sprint_blocked PASSED                       [ 48%]
tests/test_merge_engine.py::TestMergeEngine::test_dev_to_master_blocked PASSED                       [ 50%]
tests/test_merge_engine.py::TestMergeEngine::test_sprint_to_master_blocked PASSED                    [ 51%]
tests/test_merge_engine.py::TestMergeEngine::test_clean_merge_succeeds PASSED                        [ 53%]
tests/test_merge_engine.py::TestMergeEngine::test_merge_brings_commits PASSED                        [ 55%]
tests/test_scenarios.py::TestScenario01::test_clean_master_to_sprint PASSED                          [ 57%]
tests/test_scenarios.py::TestScenario02::test_clean_master_to_dev PASSED                             [ 59%]
tests/test_scenarios.py::TestScenario03::test_clean_sprint_to_dev PASSED                             [ 61%]
tests/test_scenarios.py::TestScenario04::test_dev_to_sprint_blocked PASSED                           [ 63%]
tests/test_scenarios.py::TestScenario05::test_dev_to_master_blocked PASSED                           [ 65%]
tests/test_scenarios.py::TestScenario06::test_sprint_to_master_blocked PASSED                        [ 67%]
tests/test_scenarios.py::TestScenario07::test_sprint_out_of_order PASSED                             [ 69%]
tests/test_scenarios.py::TestScenario08::test_master_out_of_order PASSED                             [ 71%]
tests/test_scenarios.py::TestScenario09::test_source_priority_resolution PASSED                      [ 73%]
tests/test_scenarios.py::TestScenario10::test_target_priority_resolution PASSED                      [ 75%]
tests/test_scenarios.py::TestScenario11::test_merge_abort_on_unresolvable PASSED                     [ 76%]
tests/test_scenarios.py::TestScenario12::test_backup_and_rollback PASSED                             [ 78%]
tests/test_scenarios.py::TestScenario13::test_scheduler_two_cycles PASSED                            [ 80%]
tests/test_scenarios.py::TestScenario14::test_multi_violation_report PASSED                          [ 82%]
```


```
tests/test_merge_engine.py::TestMergeEngine::test_sprint_to_master_blocked PASSED                    [ 51%]
tests/test_merge_engine.py::TestMergeEngine::test_clean_merge_succeeds PASSED                        [ 53%]
tests/test_merge_engine.py::TestMergeEngine::test_merge_brings_commits PASSED                        [ 55%]
tests/test_scenarios.py::TestScenario01::test_clean_master_to_sprint PASSED                          [ 57%]
tests/test_scenarios.py::TestScenario02::test_clean_master_to_dev PASSED                             [ 59%]
tests/test_scenarios.py::TestScenario03::test_clean_sprint_to_dev PASSED                             [ 61%]
tests/test_scenarios.py::TestScenario04::test_dev_to_sprint_blocked PASSED                           [ 63%]
tests/test_scenarios.py::TestScenario05::test_dev_to_master_blocked PASSED                           [ 65%]
tests/test_scenarios.py::TestScenario06::test_sprint_to_master_blocked PASSED                        [ 67%]
tests/test_scenarios.py::TestScenario07::test_sprint_out_of_order PASSED                             [ 69%]
tests/test_scenarios.py::TestScenario08::test_master_out_of_order PASSED                             [ 71%]
tests/test_scenarios.py::TestScenario09::test_source_priority_resolution PASSED                      [ 73%]
tests/test_scenarios.py::TestScenario10::test_target_priority_resolution PASSED                      [ 75%]
tests/test_scenarios.py::TestScenario11::test_merge_abort_on_unresolvable PASSED                     [ 76%]
tests/test_scenarios.py::TestScenario12::test_backup_and_rollback PASSED                             [ 78%]
tests/test_scenarios.py::TestScenario13::test_scheduler_two_cycles PASSED                            [ 80%]
tests/test_scenarios.py::TestScenario14::test_multi_violation_report PASSED                          [ 82%]
```


```
tests/test_scenarios.py::TestScenario15::test_full_end_to_end PASSED                                 [ 84%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_up_to_date PASSED                               [ 86%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_success PASSED                                  [ 88%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_blocked_upward PASSED                           [ 90%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_creates_backup PASSED                           [ 92%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_no_backup_when_disabled PASSED                  [ 94%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_with_conflicts_resolved PASSED                  [ 96%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_logs_to_db PASSED                               [ 98%]
tests/test_sync_engine.py::TestSyncEngine::test_hierarchy_sync PASSED                                [100%]

============================================= 52 passed in 53.37s =============================================
```

Conclusion and Future Scope

Conclusion

• **Automated Synchronization:**
System successfully automates branch sync across release, sprint and development.

• **Reduced Conflicts:**
Early detection and controlled merging help in minimising merge conflicts.

• **Better Code Consistency:**
Ensures all branches stay updated with validated code.

• **Improved Efficiency:**
Reduces manual effort and improves overall development workflow.

Future Scope

• **CI/CD Integration:**
Can be integrated with tools like Jenkins or GitHub Actions.

• **Smart Conflict Handling:**
AI-based prediction and resolution can be added.

• **Dashboard Development:**
Real-time visualisation of logs and violations.

• **Multi-Repo Support:**
Extend the system for large-scale enterprise projects.

References

1. *Challenges of Resolving Merge Conflicts: A Mining and Survey Study*
Gustavo Vale, Claus Hunsen, Eduardo Figueiredo, and Sven Apel
Available at: https://www.se.cs.uni-saarland.de/publications/docs/VHF+22.pdf

Appendices

Appendix A: Sample Configuration Parameters

The following parameters were used to configure the system for automated branch synchronization:

- Source Branch: release
- Target Branch: sprint/development
- Number of Cycles: 1–2
- Synchronization Interval: 2–24 hours
- Backup Option: Enabled
- Conflict Resolution Strategy: Source branch preference

Appendix B: Sample Log Format

The system generates logs to track synchronization activities and violations. A sample log format is shown below:

- Commit ID
- Developer Name
- Commit Message
- Branch Name
- Violation Type
- Timestamp
- Merge Status

Appendix C: Test Case Summary

The system was tested under multiple scenarios to validate its functionality:

- Normal branch synchronization
- Detection of unauthorised commits
- Merge conflict handling
- Backup branch creation
- Invalid merge direction blocking

All the screenshots for test cases, logs, and configuration parameters are attached and defined in the figures and tables page.